

Module 7 Assignment: Functions and Stored Procedures
CPT 166 – Fundamentals of SQL
Sylvia Schlotterbeck 11/16/25

1. Function: Calculate Total Sales for a Product:

I started solving this question by first creating a view that contained all the columns I needed to perform the required function (product_id, quantity, and price). I knew that functions run more quickly than views in large datasets, but I wanted to keep the code of the function as streamlined as possible, hence creating the view to tidy up the code in the function:

```
CREATE OR REPLACE VIEW vw_product_sales AS
    SELECT order_details.product_id, product_name, quantity, price
    FROM order_details
    LEFT JOIN products ON products.product_id = order_details.product_id
;
```

I then wrote the function. This function takes in one integer parameter (x_product_id), creates a local numeric variable (total_sales) whose value is set at one (:=0), and then runs the aggregate function SUM() to add up the quantity * price where the product_id in the view matches the input parameter, then places that sum into the local variable, and returns that local variable total_sales as the output of the function:

```
CREATE OR REPLACE FUNCTION get_product_sales(x_product_id INT)
RETURNS numeric
LANGUAGE plpgsql
AS $$
DECLARE
    total_sales NUMERIC :=0;

BEGIN
    SELECT SUM(quantity * price)
    INTO total_sales
    FROM vw_product_sales
    WHERE product_id = x_product_id;

    RETURN total_sales;
END;
$$;
```

NOTE: I originally wrote this function without declaring a local variable and didn't have success making it work, finally figuring it out after reading about declaring local variables and adding one in.

After creating the function, I ran three test queries to test its functionality. The queries and screenshots of the returned results are as follows:

```
SELECT get_product_sales(11) AS total_sales_for_product_11;
```

	total_sales_for_product_11 
1	14826.00

```
SELECT get_product_sales(72) AS total_sales_for_product_72;
```

	total_sales_for_product_72 
1	28048.80

```
SELECT get_product_sales(65) AS total_sales_for_product_65;
```

	total_sales_for_product_65 
1	15682.25

2. Function: Count Orders for a Customer:

For this question, I didn't create a view first, since all of the columns I needed to create the required function already existed in a single table (orders).

The function I wrote (count_orders), takes in one integer parameter (x_customer_id), creates a local integer variable called (customer_order_count) and assigns it a starting value of 0 (:=0), then selects the aggregate function COUNT() to count the number of orders that the customer given in the parameter (x_customer_id) has placed according to the orders table ('FROM orders WHERE customer_id = x_customer_id'). It places the results of the COUNT() function into the local variable (customer_order_count) and then returns that local variable as the functions output at the conclusion of the function:

```
CREATE OR REPLACE FUNCTION count_orders(x_customer_id INT)
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
    customer_order_count INT :=0;
BEGIN
    SELECT COUNT(order_id)
    INTO customer_order_count
    FROM orders
    WHERE customer_id = x_customer_id;

    RETURN customer_order_count;
END;
$$;
```

After creating the function, I tested it by first running a query on a customer whose customer_id *did* show up in the orders table (the customer who has the customer_id of '90'), and as you can see from the screenshot that follows the SELECT query, it returned an order count of 8 for that customer:

```
SELECT count_orders(90) AS order_count_for_customer_90;
```

	order_count_for_customer_90 
1	8

I next tested a query on a customer_id that did not show up in the order table, and as you can see from the screenshot of the results of that query, the query returned an order count of 0, as expected:

```
SELECT count_orders(1000) AS order_count_for_customer_1000;
```

	order_count_for_customer_1000 
1	0

3. Stored Procedure: Add a New Product

NOTE: For this question, I added extra categories to the stored procedure that was outlined in the instructions so that the products I inserted in the products table would have all of their columns filled in.

In creating the function for this question, I tried to make it as succinct as possible, and experimented (successfully) with not assigning any local values, which was satisfying. This function adds a record to the products table. It takes parameters for all of the columns in the existing products table (product id, product name, category id, unit description, and price), and then inserts those values into a new row in the products table.

The line “INSERT INTO...” outlines which table and which columns the parameter values should be inserted into, and the “VALUES...” line describes which parameters should go into which columns (I ordered the parameters in the function in the same order as the columns in the products table to keep things as clear as possible).

I added a “RAISE NOTICE” line to clarify what the function did when it was successfully run:

```
CREATE OR REPLACE PROCEDURE add_product(prod_id INT, prod_name TEXT,
cat_id INT, unit_descr TEXT, price NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO products (product_id, product_name, category_id, unit,
price)
    VALUES (prod_id, prod_name, cat_id, unit_descr, price);

    RAISE NOTICE 'New product added to products table';
END;
$$;
```

After creating the Stored Procedure, I then called it three times to test it out. What follows is the three calls I made, and screenshots, the first demonstrating the success of the “Raise Notice” line in the stored procedure when it is called successfully, the second showing the newly inserted products in the products table:

```
CALL add_product(78, 'Willy Wonka Everlasting Gobstoppers', 3, '50 gal. barrel', 450.50);
```

```
CALL add_product(79, 'Gustavo Glorious Grape Juice', 1, '12 - 12-oz. bottles', 11.50)
```

```
CALL add_product(80, 'Stinky Socks Limburger', 4, '6 - 8-oz. sock-shaped packages', 33.33)
```

1
CALL add_product(80, 'Stinky Socks Limburger ', 4, '6 - 8-oz. sock-shaped packages', 33.33);

Data Output
Messages
Notifications

NOTICE: New product added to products table
CALL

Query returned successfully in 39 msec.

	product_id [PK] integer	product_name character varying (255)	category_id integer	unit character varying (255)	price numeric (10,2)
63	63	Vegie-spread	2	15 - 625 g jars	43.90
64	64	Wimmers gute Semmelknadel	5	20 bags x 4 pieces	33.25
65	65	Louisiana Fiery Hot Pepper Sauce	2	32 - 8 oz bottles	21.05
66	66	Louisiana Hot Spiced Okra	2	24 - 8 oz jars	17.00
67	67	Laughing Lumberjack Lager	1	24 - 12 oz bottles	14.00
68	68	Scottish Longbreads	3	10 boxes x 8 pieces	12.50
69	69	Gudbrandsdalsost	4	10 kg pkg.	36.00
70	70	Outback Lager	1	24 - 355 ml bottles	15.00
71	71	Flotemysost	4	10 - 500 g pkgs.	21.50
72	72	Mozzarella di Giovanni	4	24 - 200 g pkgs.	34.80
73	73	Red Kaviar	8	24 - 150 g jars	15.00
74	74	Longlife Tofu	7	5 kg pkg.	10.00
75	75	Rhenbreu Klosterbier	1	24 - 0.5 l bottles	7.75
76	76	Lakkalikeeri	1	500 ml	18.00
77	77	Original Frankfurter grune Soae	2	12 boxes	13.00
78	78	Willy Wonka Everlasting Gobstoppers	3	50 gal. barrel	450.50
79	79	Gustavo Glorious Grape Juice	1	12 - 12 oz. bottles	11.50
80	80	Stinky Socks Limburger	4	6 - 8-oz. sock-shaped packages	33.33

Short explanation of when I would use a function versus a stored procedure:

The first differentiation I would make is that I would use a function when I wanted to create a stored function to *retrieve* data from a database, and a stored procedure when I wanted to create a stored function to help me *modify* data from a database. The next I would make would be that I would generally use a function for when I wanted a single result to be returned, and a stored procedure when I was looking for multiple results. Functions can be made that can then form building blocks of a stored procedure (can be CALLED in a stored procedure), but not the other way around, so I would utilize functions to assist when I had a more complex procedure I performed regularly that I wanted to make a stored procedure for, and then make functions that helped make the functionality of the stored procedure easier to write, read, debug, and/or modify in the future.